

Course Project Description

In this project, each group needs to apply the knowledge learned in the course to develop a command-line-based Personal Information Management (PIM) system (hereinafter referred to as “the system”). Appendix B lists the user stories the system should support.

Group Forming: Form your groups of 3 to 4 students from the same or different class sessions on Canvas before 9:00, 28th September 2026 (Monday). Afterward, students without a group will be randomly assigned to groups, and requests for group changes will be approved only if written agreements from all members of the groups involved are submitted before 9:00, 12th October 2026 (Monday). For instance, if a student S wants to change from group G1 to group G2 after 9:00, 28th September, S needs to submit written agreements from all members of groups G1 and G2.

Object-oriented vs. Procedural-oriented: You are suggested to design and implement the game in an object-oriented way because OO techniques often help produce software systems with better quality in terms of, e.g., code cohesion and coupling, understandability, and maintainability. That, however, does not mean you will be penalized for choosing the other way. We will consider the capabilities and limitations of each design and programming paradigm when grading, so if you feel more comfortable working with the procedure-oriented paradigm, or if you don't have enough time to learn the OO techniques for this project, feel free to adopt the procedure-oriented way. Particularly, you can program the game in either Java or Python. If you want to use another programming language, consult your instructor first.

Deliverables: Each group needs to submit the following deliverables, compressed into a single ZIP archive, by 20:00, 20th November 2026 (Friday). Unless explicitly stated otherwise, all the required documents in the deliverables should be in DOC, DOCX, or PDF format.

1. Software requirements specification (SRS). (6 points)

You may decide on the specifics of how users will interact with the system, but all the user stories given in Appendix B should be supported. Your requirements should be valid, consistent, complete, realistic, and verifiable.

- a. *Document structure:* See slides “The Structure of A Requirements Specification (1) & (2)” from “Lecture 04: Requirements Engineering” for the overall structure of a requirements specification; Note that chapters “System models”, “System evolution”, “Appendix”, and “Index” are not needed in your requirements specification.
- b. *Contents:* All the important functional and non-functional requirements should be included.
 - See slides “Natural Language Specification” and “Example Natural Language Specifications” from the same lecture for guidelines on and examples of writing natural language requirements.
 - To facilitate unit testing (see Deliverable 4), you should put all the code that belongs to the model of the system into a separate package named “model”. You may refer to the Model-View-Controller (MVC) pattern ^{a)} to help you understand what constitutes the model of the system. We will discuss the MVC pattern in “Lecture 6: Architectural Design”.
 - It is not recommended to add additional features, e.g., a GUI or support for online use, to the system. Your efforts to design and implement the additional features will not get you extra credit.

2. Design document. (5 points)

The deliverable should contain 1) a diagram to describe the system’s architecture, 2) diagrams to show both the structure of and the relationship among the main code components of the system, and 3) a diagram to outline the process of an example use where a user searches for and updates some records in the system.

- a. *Architecture:* Explain which generic architectural pattern you picked for the system, why you chose this pattern, and how the components of the pattern were

^{a)} <https://en.wikipedia.org/wiki/Model-view-controller>

instantiated in the context of the system. (“Architectural Design” will be discussed in Lecture 6.)

- b. *Structure of and relationship among main code components*: If you adopt object-oriented techniques, explain the classes in your design and their fields and public/protected methods; If you adopt procedural-oriented techniques, explain the user-defined data types and functions in your design. For each method/function, the explanation should include information about the return type, the method/function name, the argument names and types, and possible exception declarations. Explain how the major code components are related to each other in the system.
- c. *Example use*: Demonstrate the collaboration among the code components using a sequence or activity diagram.
- d. *Necessary textual explanations*: Each diagram should be accompanied by necessary textual explanations to facilitate understanding.

3. Implementation. (6 points)

The deliverable should include 1) the complete source code for the system, 2) a developer manual explaining how to compile the code and build the project, 3) a user manual describing how to use the system, 4) a short video (no more than 4 minutes) of the system in use, and 5) a short report on requirements coverage achieved by your implementation.

- a. *Source code*: The code implementing the model of the system should be placed in a separate package named “model”, and the other major code components should also be easily identifiable in the source code. Your implementation should only invoke Java/Python standard libraries.
- b. *Developer manual*: Here, you may assume the readers know how to properly set up a Java/Python development environment, and you don’t have to explain that in the manual. You only need to prepare the manual for one platform, e.g., Windows, macOS, or Ubuntu. You need to explain which version of JDK/Python was used for the development, which IDE should be used to open your project, which commands (and parameters, if any) should be used to build your project, and/or how to launch the system in the debugging mode.
- c. *User manual*: In the user manual, you need to explain things like which input commands are supported, what their formats are, what happens if an input command is invalid, and how to interpret the output of the system. Everything a user needs to know to use the system effectively should be explained in the user manual.
- d. *Video*: The video should be in MP4 format. Given the video’s limited length, you only need to demonstrate the system’s most important features. A feature can be important for the user because it is a basic operation or it greatly influences the user’s experience, and it can be important for you, as the developer, because, e.g., you are proud of the design behind the feature. You don’t need to explain everything in the video verbally, since most operations should be straightforward, but feel free to add narration and/or text to describe anything less obvious or worth extra attention.
- e. *Requirements coverage report*: Report in a table which user stories from Appendix B and which requirements in the SRS have/have not been implemented. Put the report in the root folder of your source code.

4. Unit tests for the system model. (4 points)

Each unit test (that’s right, you only need to write unit tests in this project) should clearly state the functionalities it exercises (e.g., using comments) and the expected results (e.g., using assertions).

- a. *Unit testing framework*: All the unit tests should be automatically executable and should pass when executed against your system implementation. It is strongly recommended that you prepare the tests using unit testing frameworks. Two example unit tests, in JUnit for Java and unittest for Python, respectively, are given in Figure 1 for your reference, but feel free to use other unit testing frameworks if you so desire. Please refer to the corresponding documents and/or tutorials for

information on writing unit tests using different unit testing frameworks.

<pre>// JUnit for Java ... public class JUnitProgram { @Test public void test_JUnit() { String str1="testcase "; assertEquals("testcase ", str1); } }</pre>	<pre># unittest for Python import unittest class Testing(unittest.TestCase): def test_string(self): a = 'some' b = 'some' self.assertEqual(a, b) if __name__ == '__main__': unittest.main()</pre>
---	---

Figure 1. Example unit tests in JUnit for Java and unittest for Python, respectively.

- b. **Test coverage:** Report the line coverage achieved by your tests on the system model in a separate file and place the report in the root folder of your source code.

5. Presentation slides and recording (4 points)

The presentation should not exceed 5 minutes. You may include screen snapshots or short video clips of the system in use in your presentation, but do not let them dominate or consume excessive presentation time.

Each group member must present for at least 1 minute. All presenters must show their student ID cards and faces at the beginning of their presentation, and the current presenter's face must always be visible in the video. Each group member who does not show their face or student ID card during the presentation will have half of their marks deducted.

a. *Presentation contents:*

- Three system requirements regarding 1) the creation of a PIR, 2) the definition of a search criterion, and 3) the search for specific PIRs, respectively.
- The overall design of the system, i.e., the combination of Deliverables 2.a and 2.b.
- One important lesson learned from the project regarding requirements engineering, API design, or unit testing.

- b. *Presentation slides and recording:* The slides should be in PDF format, and the recording should be in MP4 format.

Use of GenAI tools. Using GenAI tools in the project is allowed, but you should properly acknowledge all content in your deliverables that was created with their help. More specifically, each group must fill out the "Honour Declaration for Group Project" and include it within the final submission, even if the group has not used GenAI in the project. You may refer to <https://www.polyu.edu.hk/ar/students-in-taught-programmes/use-of-genai/> for more guidance on the use of GenAI. The "Honour Declaration" should be placed in the root folder of the ZIP archive. Failure to submit the "Honour Declaration" or submitting a false declaration will result in a penalty of up to 30% of the total points.

Grading. Each group needs to reach a consensus on the percentage contribution each member has made to the project and clearly state those percentages in the "Honour Declaration for Group Project." Your individual score for the project will be calculated based on your group's overall project score and your percentage contribution to the project. Suppose the overall project is worth x points, the group has y members ($3 \leq y \leq 4$), and one member's contribution is $z\%$. The member's individual score for the project is calculated as $\min(x * y * z\%, x)$. We encourage equal contribution. If your group has difficulty reaching such a consensus, you should submit all other required materials and report your case to the instructor before the deadline.

Appendix A. Grading Criteria

1 is for the lowest quality, coverage, etc.; 5 is for the highest.

Software requirements specification
Deliverable completeness
Document quality (in terms of contents, structure, and writing)
Requirements quality (in terms of validity, consistency, completeness, realism, and verifiability)
Design document
Deliverable completeness
Document quality (in terms of contents, structure, and writing)
Design quality (in terms of modularity, efficiency, extendibility, and justifiability)
Diagram quality (in terms of completeness, well-formedness, and understandability)
Implementation
Deliverable completeness
Document quality (in terms of contents, structure, and writing)
Code quality (in terms of reliability, readability, error handling, and using only standard lib.)
Requirements coverage
Unit tests for the system model
Deliverable completeness
Test quality (in terms of readability and granularity)
Test coverage
Presentation
Deliverable completeness
Presentation content (in terms of requirements satisfaction, design, and lessons learned)
Presentation delivery (in terms of fluency and timing)

Appendix B. User Stories for The Personal Information Management (PIM) System

- US1. As a user, I want to create different types of personal information records (PIRs) in the PIM so that all the information I care about can be managed in a single location.
- US2. As a user, I want to create new plain texts as PIRs so that I can use the PIM to take quick notes.
- US3. As a user, I want to create new tasks with the corresponding descriptions and deadlines as PIRs so that I can use the PIM to manage my to-dos.
- US4. As a user, I want to create new events with the corresponding descriptions, starting times, and alarms as PIRs so that I can use the PIM to manage my schedule.
- US5. As a user, I want to create new contacts with the corresponding names, addresses, and mobile numbers as PIRs so that I can use the PIM to manage my contacts.
- US6. As a user, I want to modify the data in existing PIRs so that I can keep the PIRs up to date.
- US7. As a user, I want to search for PIRs based on criteria concerning their types and the data stored in their fields. A criterion may check whether a piece of text (stored in a note, a description, a name, an address, or a mobile number) contains a string, whether a time (stored in a deadline, a starting time, or an alarm) is before (<), after (>), or equal to (=) another given point in time, or whether a condition combining multiple other conditions via logical connectors and (&&), or (||), and negation (!) is satisfied.
- US8. As a user, I want to print out detailed information about a specific PIR or all PIRs.
- US9. As a user, I want to delete a specified PIR.
- US10. As a user, I want to store the PIRs in a file with the extension name ".pim" so that I can access them using the PIM in the future.
- US11. As a user, I want to load the PIRs from a file with the extension name ".pim" so that I can continue working with the PIRs I stored earlier.